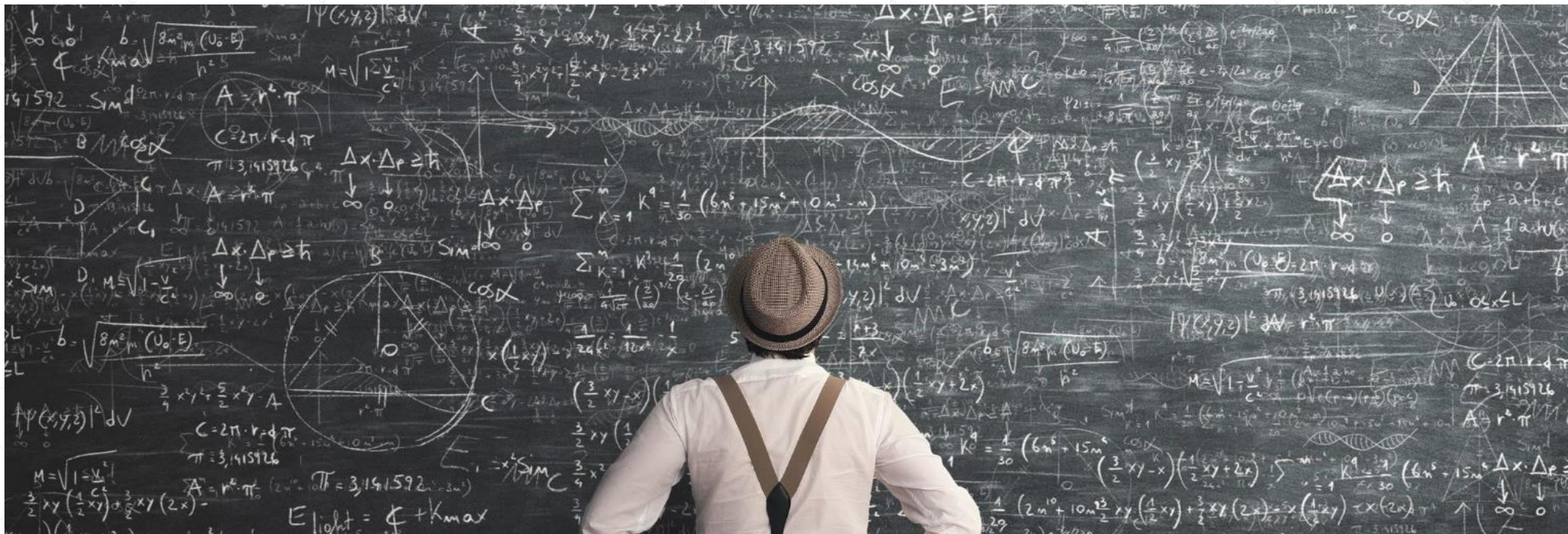




函数式编程概述

北京理工大学计算机学院
金旭亮

编程范式的演化与转移

结构化编程

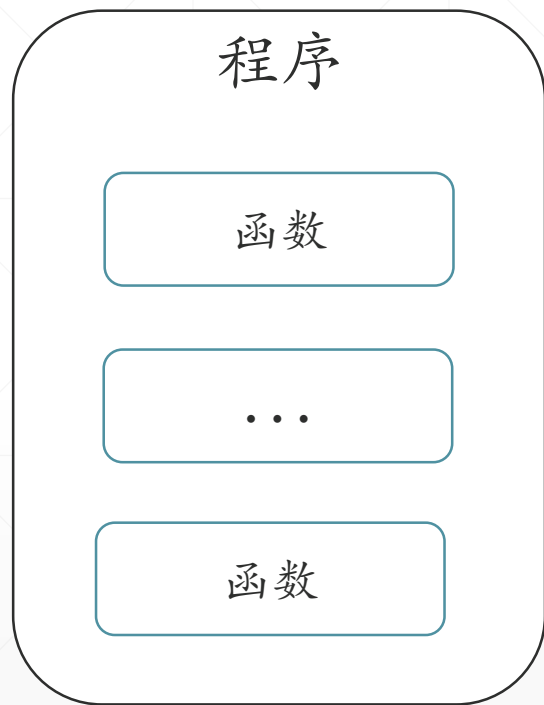
面向对象编程

函数式编程

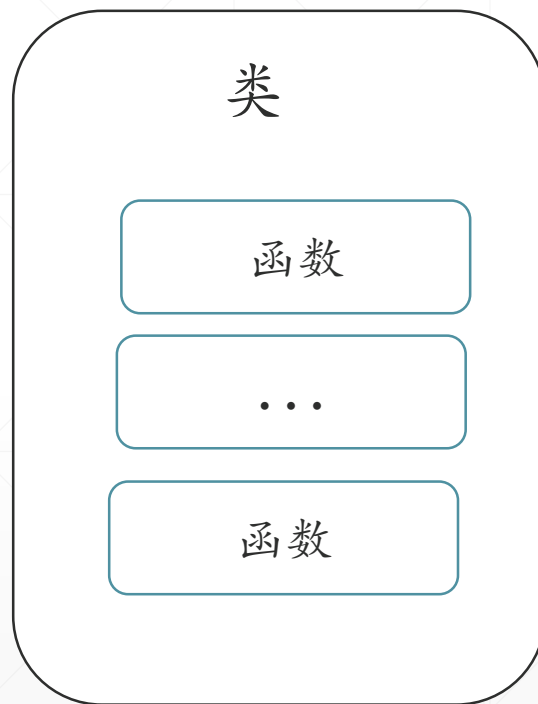
Imperative(命令式)
编程风格 (How to do)

Declarative(声明式)
编程风格 (What to do)

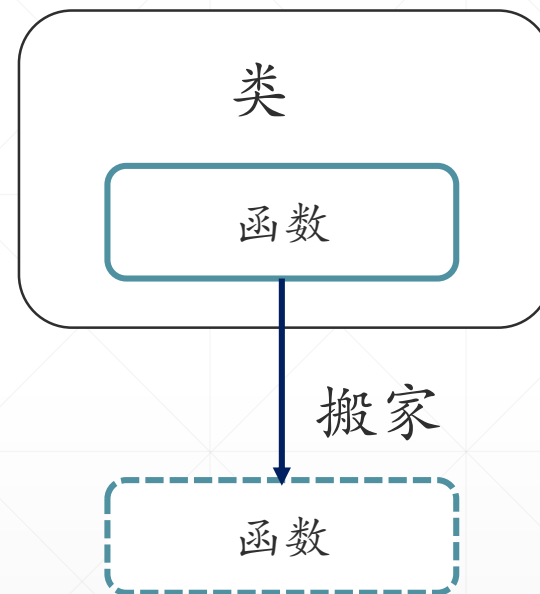
对比了解三种编程范式



结构化编程



面向对象编程



函数式编程

函数式编程

面向对象编程



Java从JDK 8开始，全面拥抱函数式编程范式.....



“函数式编程兴起”的时代背景



进入二十一世纪以来，“函数式编程”日益流行，一些新的语言，比如Kotlin/F#，从一开始就打算设计为“面向对象”+“函数式”的，而一些传统的面向对象编程语言，比如C#和Java，则在不断地将函数式编程特性引入到它的基础语法特性中。



“函数式编程”流行的原因之一，在于它具有很强的表达能力，能高效方便地开发分布式的能应对高并发挑战的软件系统。



计算机硬件技术的飞速进步，特别是现代计算机普遍配置有大的内存和多核CPU，系统资源充裕，计算能力强悍，为采用“函数式编程范式”编写出来的程序，提供了坚实的物质基础。

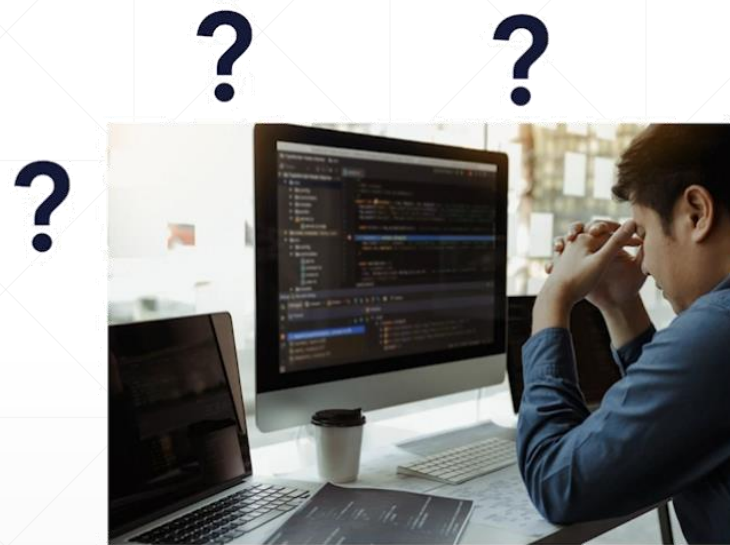
为什么现在函数式编程会流行？

原有的面向对象软件开发方式，存在着一些难以解决的痛点问题。

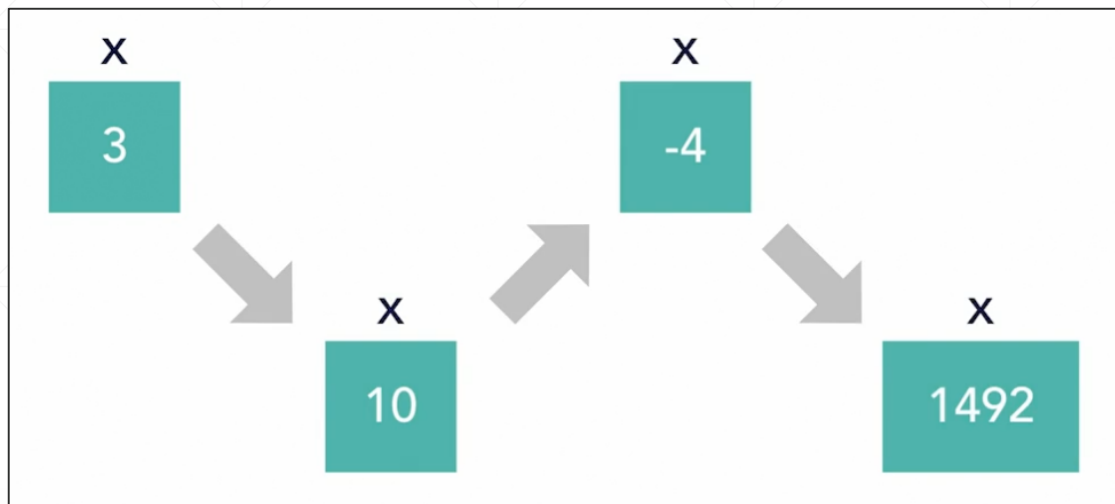
函数式编程，为解决这些痛点问题，提供了一种行之有效的解决方案.....

痛点：对象“状态可变”问题

使用面向对象思想开发出来的软件，其对象的状态往往是可以更改的，当软件变得复杂，特别是在多线程环境下，如果软件出现了Bug，要重现故障并进而消除Bug，通常是非常困难的一件事情。



变量是“值的容器”还是“值的名字”？



从面向对象编程视角来看，程序中使用的变量，其实只是一个**值的容器**，这个容器中，可以放置不同的值。

Java中的“只读”变量

```
final int x = 3;
```

从函数式编程视角，变量其实只是值的一个“**别名**”，值本身是不能改的。

函数式编程，让对象“只读”.....

面向对象代码

```
MyClass obj = new MyClass( 100 );  
//对象状态是可变的  
int newValue = obj.increase( 1 );
```

函数式风格的代码

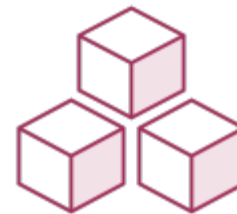
```
MyClass obj = new MyClass( 100 );  
int oldValue = obj.getValue();  
//对象是只读的，如果要改值，创建一个新对象  
MyClass newObj = new MyClass(oldValue + 1);  
int newValue = newObj.getValue();
```

采用函数式编程思想设计类，强调“类”所封装的数据，应该只被初始化一次，之后就不再更改。

JDK中的String类型，就是用“函数式编程”风格设计出来的一个例子。

让对象“只读”，就从“面向对象”向“函数式编程”前进了一步.....

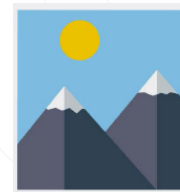
“对象只读”，简化了对象状态的管理



函数式编程强调“数据的只读”特性，将“读/写”操作明确分离，从而大大地简化了对象状态的管理。

多线程访问只读对象，无需加锁，提升了程序的性能。

痛点：代码可维护性问题



面向对象编程，主要使用“命令式”的编码风格，需要关注大量的细节，代码理解不易。

函数式编程，普遍并且推荐使用“声明式”的编码风格，表现力强，代码简洁易读。

两者PK，函数式编程有着独特的优点.....

声明式编程 vs. 命令式编程



要盖一幢房子.....

Imperative



How do I get it?

盖房子，你需先设计、准备建筑材料，然后打地基，之后才能从底向上地盖房子

Declarative



What is it?

房子需要有门、有窗，有屋顶，把它们组合起来，就得到了一间房子.....

两种编程风格的对比-1

```
static void calculateUseImperativeStyle() {  
    //定义一个变量, 用于保存中间结果  
    int sum = 0;  
    //写一个循环  
    for (int i = 1; i <= 100; i++) {  
        //每次循环时, 都将当前数字取出来, 累加到sum变量中  
        sum += i;  
    }  
    //输出最终结果  
    System.out.println(sum);  
}
```

命令式编程风格

```
static void calculateUseDeclarativeStyle() {  
    //我要求出[1,100]这个区间中所有整数的和  
    int sum = IntStream.rangeClosed(1, 100).sum();  
    System.out.println(sum);  
}
```

声明式编程风格

两种编程风格的对比-2

```
List<Integer> integerList = Arrays.asList(1,2,3,4,4,5,5,6,7,7,8,9,9);
```

```
List<Integer> uniqueList = new ArrayList<>();  
for(Integer i :integerList)  
    if(!uniqueList.contains(i)){  
        uniqueList.add(i);  
    }  
System.out.println("unique List : " + uniqueList);
```

命令式编程风格

```
List<Integer> uniqueList1 = integerList.stream()  
    .distinct()  
    .collect(toList());  
System.out.println("uniqueList1 : " + uniqueList1);
```

声明式编程风格

函数式编程，大量地使用声明式编程风格.....



声明式编程风格（Declarative Programming），比之命令式编程风格（Imperative Programming），能产生更短的更易读的代码。



Java 8引入了Lambda表达式特性，提供了Stream API，其内置的函数支持动态组合和级联调用，能够方便地实现“声明式”的编程方式。



函数式与面向对象编程的基本构造元素

面向对象编程，编程的基本单元是“类”，函数必须放在类中，从属于“类”。

函数式编程，以“函数”作为编程的基本构造块，函数之间可以相互协作和动态组合。

函数式编程能很容易地实现“行为的参数化”



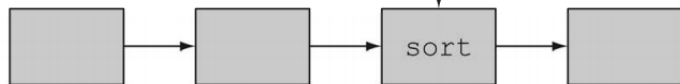
函数封装了“行为”，在函数式编程中，是“把函数作为值”来看待的。



既然“函数是值”，那么它就可以作为另一个函数的参数或返回值，在程序中“传来传去”，这个就是“行为的参数化”。

Java使用Lambda表达式来代表那些需要“传来传去”的行为，将其作为函数参数或返回值，从而实现了“行为参数化”。

```
public int compareUsingCustomerId(String inv1, String inv2){  
    ....  
}
```



函数式编程，以“函数”为核心



函数，拥有以下特性：

1

函数没有副作用

2

函数是一等公民

3

函数支持组合与级联调用

后面的课程，会详细介绍这些重要概念.....